

Considerazioni finali

Nel test di Miller–Rabin scegliamo una base a uniformemente a caso da $\{1, 2, \dots, n - 1\}$ **con rimpiazzo**. La probabilità che due basi coincidano in due scelte indipendenti è:

$$P\{a_i = a_j\} = \frac{1}{n - 1}.$$

Per un numero RSA a 2048-bit:

$$n \approx 2^{2048} \quad \Rightarrow \quad P\{a_i = a_j\} \approx 2^{-2048},$$

praticamente zero.

La probabilità è così piccola da essere minore della probabilità di qualunque evento fisico realisticamente osservabile.

Considerazioni finali

Le basi 1 e $n - 1$ sono *inutili* per la generazione di testimoni, quindi solitamente si scartano nelle implementazioni: ogni base a si sceglie uniformemente a caso da $\{2, 3, \dots, n - 2\}$.

Base 1:

$1^d \equiv 1 \pmod{n}$ e tutte le successive potenze rimangono 1: **non possiamo mai trovare un testimone.**

Base $n - 1$:

$(n - 1) \equiv -1$; le potenze producono solo 1 o -1: **anche questa base non fornirà mai un testimone.**

Nella sostanza includere o escludere 1 e $n - 1$ non cambia nulla perché la probabilità di scegliere 1 oppure $n - 1$ è comunque trascurabile.

Considerazioni finali

Conclusione.

- Inserire controlli “per evitare di scegliere due basi uguali” **non ha alcun senso.**
- È una “micro-ottimizzazione negativa”: aggiunge lavoro *per evitare un evento che non avviene.*
- Può solo **peggiore la performance.**

Considerazioni finali

Ogni round riduce la probabilità di errore a un fattore $\leq 1/4$.

Le basi devono essere *indipendenti e casuali* perché:

$$P\{\text{errore dopo } k \text{ iterazioni}\} \leq (1/4)^k.$$

Se evitassimo rimpiazzo (cioè “no ripetizioni”) introdurremmo un **inutile bias**:

→ le basi successive non sarebbero più indipendenti.

→ Non abbiamo più una valutazione pulita della probabilità di errore.

Considerazioni finali

Dopo 64 iterazioni indipendenti, la probabilità di errore del test è al più:

$$4^{-64} = 2^{-128}.$$

È la probabilità che, lanciando una moneta perfettamente equa, esca sempre “testa” per 128 lanci consecutivi.

Un evento realisticamente impossibile da osservare!

Pseudoprimi

Il test di primalità Miller–Rabin viene eseguito su una base alla volta.

Per una singola base a , un numero composto può non rivelare la sua compostezza, cioè comportarsi come se fosse primo.

Attenzione: qui ci riferiamo al risultato del test **su una singola base**, non all'intero procedimento ripetuto con basi diverse scelte a uniformemente a caso.

Abbiamo già visto che un numero composto che sembri primo su tutte le basi scelte dal test è invece un evento praticamente impossibile. La probabilità è talmente piccola da essere fisicamente irrilevante.

Pseudoprimi

Un numero **primo** passa *sempre* i test di primalità di Fermat e Miller–Rabin.

Un numero **composto** *a volte* si comporta come primo su alcune basi.

Da qui nascono due concetti fondamentali:

- **Pseudoprimo.**
- **Pseudoprimo forte.**

Attenzione: entrambi sono **relativi a una specifica base a .**

- *Pseudoprime to the base a .*
- *Strong pseudoprime to the base a .*

Pseudoprimo (per una base)

Un numero **composto** n è uno pseudoprimo rispetto alla base a ($1 < a < n$ e $\gcd(a, n) = 1$) se $a^{n-1} \equiv 1 \pmod{n}$.

In altre parole:

un numero composto n è uno **pseudoprimo per la base a** se il **test di Fermat** sulla base a dice *probabilmente primo*.

Essere pseudoprimo per una base è una condizione **necessaria** per essere primo, ma **non sufficiente**: un numero composto può soddisfare la stessa proprietà.

Pseudoprimo forte (per una base)

Se n è dispari allora $n - 1$ può essere scritto come $n - 1 = 2^r d$, con $r > 0$ e d dispari.

Un numero **composto** n **dispari** è uno **pseudoprimo forte** per la base a (con $1 < a < n$ e $\gcd(a, n) = 1$) se vale almeno una delle due condizioni:

In altre parole:

un numero composto n dispari è uno **pseudoprimo forte per la base a** se il **test di Miller-Rabin** sulla base a dice *probabilmente primo*.

Pseudoprimo forte (per più basi)

Siano $a_1, a_2, \dots, a_k$ una lista di basi.

Un intero composto n dispari è uno **pseudoprimo forte per le basi** $a_1, \dots, a_k$ se il test di Miller–Rabin restituisce “probabilmente primo” su ciascuna di queste basi, cioè nessuna di queste basi può essere usata per fornire un testimone della compostezza di n .

Per **ciascuna** base in $\{a_1, \dots, a_k\}$ vale almeno una delle condizioni:

- $a^d \equiv 1 \pmod n$.
- $a^{2^j d} \equiv -1 \pmod n$ per qualche $0 \leq j < r$,

dove $n - 1 = 2^r d$, con $r > 0$ e d dispari.

Definizione di ψ_k

Siano $p_1 = 2, p_2 = 3, p_3 = 5, \dots, p_k$ i primi k numeri primi.

$$\psi_k = \min\{n: n \text{ è pseudoprimo forte per tutte le basi } p_1, \dots, p_k\}.$$

ψ_k è il più piccolo composto che **inganna MR** su *tutte* le prime k basi prime.

⋮

Da ψ_k a MR deterministico

Se conosciamo ψ_k , possiamo costruire una **versione deterministica (sempre corretta)** di Miller–Rabin per testare la primalità di tutti i numeri minori di ψ_k .

Come?

$\psi_1 = 2047$ implica che tutti gli interi composti $n < 2047$ non sono pseudoprimi forti per la base 2. In altre parole: è sufficiente usare la sola base $a = 2$ per verificare deterministicamente (senza errori) la primalità di tutti i numeri < 2047 .

$\psi_4 = 3215031751$ implica che per tutti i numeri minori di 3215031751 abbiamo una versione deterministica (infallibile) che prova MR sulle sole basi 2, 3, 5, 7.

Da ψ_k a MR deterministico

Se conosciamo ψ_k , possiamo costruire una **versione deterministica (sempre corretta)** di Miller–Rabin per testare la primalità di tutti i numeri minori di ψ_k .

Regola generale.

Se $n < \psi_k$, allora MR sulle basi $\{2, 3, 5, \dots, p_k\}$ è **esatto al 100%**:

Per ciascun numero composto $n < \psi_k$ esiste almeno una base in $\{2, 3, 5, \dots, p_k\}$ che lo “smaschera”.

Siccome $\psi_{12} > 2^{64}$, allora:

se $n \leq 2^{64} = 18\,446\,744\,073\,709\,551\,616$, è sufficiente testare $a = 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31$, e 37.

Da ψ_k a MR deterministico

La versione deterministica del test di Miller–Rabin **non richiede affatto che le basi siano numeri primi**.

Le basi prime sono state utilizzate **solo nei primi lavori**, per motivi tecnici.

Usando basi prime, se un numero composto inganna MR in base a , allora *spesso* inganna MR anche su almeno un **fattore primo** di a ;

Quindi lavorare *solo* con basi prime spesso riduce la ricerca e semplifica la struttura degli pseudoprimi.

Da ψ_k a MR deterministico

Molti dei set “ottimali” noti **NON** sono insiemi di primi.

Esempio.

Per tutti gli interi a 64 bit, basta usare le seguenti 7 basi:

$\{2, 325, 9375, 28178, 450775, 9780504, 1795265022\}$.

MR deterministico solo con upper bound

Il test Miller–Rabin diventa **deterministico solo se** si conosce in anticipo un upper bound B sui numeri da testare, e si usano basi fisse appropriate, determinate tramite ricerche empiriche al computer.

Per numeri tipici della crittografia (1024–2048–4096 bit) non esistono liste di basi fisse affidabili per questi range enormi.

Conclusione:

Per numeri molto grandi (come nei contesti RSA), Miller–Rabin viene sempre usato in forma randomizzata, con probabilità di errore realisticamente zero.

Implementazione di Miller-Rabin

Quando vogliamo testare la primalità di numeri molto grandi (2048 bit, 4096 bit, oppure più), **i tipi standard del C non bastano**.

uint64_t arriva solo a $2^{64} - 1 \rightarrow$ *assolutamente insufficiente per RSA*

Dobbiamo gestire interi della forma:

$$n \approx 2^{2048}$$

Serve quindi una libreria che gestisca **interi a precisione arbitraria**.

GMP (GNU Multiple Precision Library) è lo standard de facto.

Implementazione di Miller-Rabin

GMP (GNU Multiple Precision Arithmetic Library): una libreria open-source che permette di lavorare con **interi di dimensione arbitraria** (centinaia, migliaia... anche milioni di bit); fornisce funzioni semplici per: addizione, sottrazione, moltiplicazione, modulo, esponenziazione modulare, generazione di numeri casuali di grandi dimensioni; è molto veloce ed è la libreria standard usata in molti progetti accademici.

Software crittografici “reali” (OpenSSL, libsodium, ecc.) usano altre librerie interne.

Ma per *capire e sperimentare* l'algoritmo, **GMP è perfetta**: semplice, potente e già pronta all'uso.

Tipi `mpz_t` e libreria GMP

`mpz_t` è il tipo di dati GMP che rappresenta **interi arbitrariamente grandi**.

Internamente, un `mpz_t` è una struttura con:

- `_mp_d` : array dinamico di “limb” (blocchi a 64 bit).
- `_mp_size` : quanti limb sono usati (segno del numero incluso).
- `_mp_alloc`: quanti limb sono allocati.

In pratica:

- Un intero `mpz_t` è un **array dinamico di word a 64 bit**.
- Quando il numero cresce, GMP rialloca automaticamente.

Rappresentazione degli interi mpz_t

Gli interi sono rappresentati in base 2^{64}

Esempio.

Come è memorizzato il numero $n = 2^{256} + 123$?

$$n = 1 \cdot 2^{64 \cdot 4} + 0 \cdot 2^{64 \cdot 3} + 0 \cdot 2^{64 \cdot 2} + 0 \cdot 2^{64 \cdot 1} + 123$$

Cioè un numero con:

limb 1 = 123

limb 2 = 0

limb 3 = 0

limb 4 = 0

limb 5 = 1

Aritmetica estremamente veloce anche su numeri enormi.

Rappresentazione degli interi mpz_t

Tutte le variabili GMP **devono essere inizializzate**:

```
mpz_t x;    // dichiarazione  
mpz_init(x);    // x = 0  
mpz_clear(x);   // deallocazione
```

Valori iniziali:

```
mpz_init_set_ui(x, 17);  
mpz_set_str(x, "12345678901234567890", 10);
```

Operazioni di base (usate in MR)

Aritmetica

```
mpz_add(r, a, b);    //  $r = a + b$   
mpz_sub(r, a, b);    //  $r = a - b$   
mpz_mul(r, a, b);    //  $r = a * b$   
mpz_mod(r, a, n);    //  $r = a \bmod n$ 
```

Esponenziazione modulare

```
mpz_powm(r, a, e, n); //  $r = a^e \bmod n$ 
```

Operazioni di base (usate in MR)

Parità / divisione per 2

`mpz_even_p(x);`

`mpz_fdiv_q_2exp(d, x, c); //  $d = x / (2^c)$`

`mpz_fdiv_q_2exp(d, x, 1);`

→ calcola $d = \lfloor x/2^1 \rfloor$.

Nel test di Miller–Rabin, serve per estrarre i fattori 2 da $n - 1$ in modo efficiente (shift).

GCD (per rilevare testimoni triviali)

`mpz_gcd(g, a, n); // calcola  $g = \gcd(a, n)$  cioè il massimo comun divisore di  $a$  e  $n$ .`

Generazione di basi casuali

Generatore GMP

```
gmp_randstate_t state; // dichiarazione di una variabile state il cui tipo è gmp_randstate_t
gmp_randinit_default(state); // inizializza lo stato RNG
gmp_randseed_ui(state, time(NULL)); // imposta il seed da clock
```

`gmp_randstate_t` è un **tipo definito da GMP** (con un typedef), che incapsula internamente:

- il tipo di generatore (MT, default, ecc.);
- il seed;
- lo stato interno del PRNG.

Generazione di basi casuali

Generatore GMP

```
gmp_randstate_t state; // dichiarazione di una variabile state il cui tipo è gmp_randstate_t
gmp_randinit_default(state); // inizializza lo stato RNG
gmp_randseed_ui(state, time(NULL)); // imposta il seed da clock
```

Generare un valore uniforme in $[0, m-1]$

```
mpz_urandomm(a, state, m);
```

Per ottenere un valore uniforme in $\{2, \dots, n - 2\}$

```
mpz_urandomm(a, state, n_minus_3);
mpz_add_ui(a, a, 2);
```

Moltiplicazione modulo n

```
/*-----  
* Moltiplicazione modulo n  
*  $x = (a * b) \bmod n$   
* (con GMP mul è già big-int safe)  
*-----*/
```

```
static void mulmod(mpz_t x, const mpz_t a, const mpz_t b, const mpz_t n) {  
    mpz_mul(x, a, b);  
    mpz_mod(x, x, n);  
}
```

Esponenziazione modulare

```
/*-----  
* Esponenziazione modulare  
*  $x = a^e \bmod n$   
*  
* Corrisponde al calcolo iniziale  $x = a^d \bmod n$   
*-----*/
```

```
static void powmod(mpz_t x, const mpz_t a, const mpz_t e, const mpz_t n) {  
    mpz_powm(x, a, e, n); // GMP ha già l'esponenziazione modulare efficiente  
}
```

Testimoni

```
/*  
 * cerca_testimone_MR(a, n)  
 *  
 * Restituisce:  
 * 1 se da 'a' si trova un testimone di COMPOSTEZZA di n  
 * 0 altrimenti  
 */  
static int cerca_testimone_MR(const mpz_t a, const mpz_t n) {  
  
    ...  
  
}
```

Funzione principale

```
/*-----  
* Versione randomizzata di Miller–Rabin.  
*  
* miller_rabin(n, k, state)  
*  
*     n: intero da testare  
*     k: numero di round (basi casuali)  
*     state: stato RNG GMP  
*  
* Restituisce:  
* 1 se n è "probabilmente primo"  
* 0 se n è sicuramente composito  
*-----*/  
  
int miller_rabin(const mpz_t n, int k, gmp_randstate_t state) {  
    ...  
}
```

Funzione principale

```
int miller_rabin(const mpz_t n, int k, gmp_randstate_t state) {  
    // Gestione casi triviali  
    if (mpz_cmp_ui(n, 2) < 0) return 0; // n < 2 --> non primo  
    if (mpz_cmp_ui(n, 2) == 0) return 1; // 2 è primo  
    if (mpz_even_p(n)) return 0; // pari > 2 --> composto  
  
    //dichiarazioni e inizializzazioni  
    mpz_t a, n_meno_3, g;  
    mpz_inits(a, n_meno_3, g, NULL);  
  
    // n_meno_3 = n - 3;  
    mpz_sub_ui(n_meno_3, n, 3);  
  
    ...  
}
```

Funzione principale

```
for (int i = 0; i < k; i++) { // k iterazioni
    // Scegli a uniformemente in {2, ..., n-2}
    mpz_urandomm(a, state, n_meno_3);
    mpz_add_ui(a, a, 2);

    // Caso triviale: se gcd(a, n) > 1 --> n composto
    mpz_gcd(g, a, n);
    if (mpz_cmp_ui(g, 1) > 0) {
        mpz_clears(a, n_meno_3, g, NULL);
        return 0; // composto (gcd-testimone)
    }

    // Cerca testimone con base a
    if (cerca_testimone_MR(a, n)) {
        mpz_clears(a, n_meno_3, g, NULL);
        return 0; // composto (testimone-Fermat o testimone-radice)
    }
}
```

Funzione principale

```
    // all'uscita del for  
    mpz_clears(a, n_meno_3, g, NULL);  
    return 1; // probabilmente primo (errore < 4^{-k})  
}
```


Funzione cerca testimoni

```
static int cerca_testimone_MR(const mpz_t a, const mpz_t n) {  
    mpz_t d, x, n_meno_1;  
    mpz_inits(d, x, n_meno_1, NULL);  
  
    // n-1  
    mpz_sub_ui(n_meno_1, n, 1);  
    mpz_set(d, n_meno_1);  
  
    // Decomposizione  $n-1 = 2^r * d$  con d dispari  
    int r = 0;  
    while (mpz_even_p(d)) {  
        mpz_fdiv_q_2exp(d, d, 1); //  $d = d / 2$   
        r++;  
    }  
    ... // Vedere il file per il frammento rimanente
```

Divertimenti

Per divertimento, usare/modificare l'implementazione di Miller–Rabin per le seguenti esplorazioni.

- Cercare i valori di ψ_k per k piccoli.
- Fino a che k riuscite ad arrivare con un PC moderno, prima che la ricerca diventi troppo lenta?
- Provare insiemi diversi di basi, non necessariamente prime, per vedere come cambia la resistenza ai numeri composti.